THE STABLECOIN COMMERCE STACK CHALLENGE · IGNYTE X CIRCLE X ARC · 2026

Float

Float your invoices. SMEs get paid today, not in 90 days.

TRACK

Track 2: SME Trade Finance & Working Capital

LIVE APPLICATION

float-arc.vercel.app

CIRCLE ACCOUNT

coinlistx100@gmail.com

GITHUB

github.com/PigAssasin/float-arc

SETTLEMENT LAYER

Arc Testnet (Chain ID 5042002)

SUBMISSION DEADLINE

July 13, 2026

1. Problem & Solution

Small and medium enterprises (SMEs) are the backbone of global trade, yet they routinely wait 30 to 90 days to be paid after delivering goods or services. This working capital gap (often exceeding a company's monthly payroll) forces SMEs to take on expensive debt, turn down new contracts, and miss growth opportunities. Traditional invoice factoring exists, but it is slow, paper-heavy, and accessible only to well-established businesses with banking relationships.

Float eliminates this gap with a transparent, on-chain invoice factoring protocol. Sellers receive 75 to 88% of an invoice's face value immediately in USDC. Buyers pay 100% at maturity. The spread flows to a decentralized liquidity pool whose investors earn 12 to 25% yield. Every party (seller, buyer, investor) interacts through a single web application, with no intermediary banks or manual underwriting.

SELLER (SME)

Creates an invoice, receives 75–88% in USDC immediately. A stake is withheld as recourse and returned when the buyer pays.

BUYER

Approves the invoice, locks collateral, and pays 100% at due date. A 7-day grace period applies before default can be triggered.

INVESTOR

Deposits USDC, receives fLP tokens. Share value rises as fees accrue. Withdrawable at any time.

2. Products Used

Circle: User-Controlled Wallets (W3S SDK)

Most SME owners are not crypto-native. Requiring MetaMask installation is a hard drop-off point. Float integrates Circle's W3S SDK as a first-class connection option, creating a self-custodied wallet secured by a PIN with no seed phrase, no browser extension, and no prior crypto knowledge required.

Endpoint / Method	Purpose
`POST /v1/w3s/users`	Idempotent user creation; 409 means already exists, not an error
`POST /v1/w3s/users/token`	Generates userToken + encryptionKey for the session
`POST /v1/w3s/user/wallets`	Creates wallet, returns challengeId for PIN setup
`GET /v1/w3s/wallets`	Fetches live wallet address after PIN is set
`sdk.execute(challengeId, cb)`	Renders Circle's embedded PIN UI for challenge signing

App ID: `f06cb713-a2a3-57d2-9662-13607ec5f12f` | SDK: `@circle-fin/w3s-pw-web-sdk`

Circle: USDC

All advances, collateral deposits, seller stakes, repayments, and pool capital are denominated in USDC. The native USDC contract on Arc Testnet (`0x3600000000000000000000000000000000000000`) is used directly; no wrapping or bridging is required. Because Arc uses USDC as the gas token, users never need to acquire a separate asset just to pay transaction fees.

Arc Testnet

Float is deployed 100% on Arc Testnet (Chain ID 5042002). Arc's EVM compatibility allowed the team to use standard Solidity, Hardhat, wagmi v2, and viem tooling with zero changes. The network's USDC-as-gas design is uniquely suited to stablecoin finance applications: every on-chain action (creating invoices, locking collateral, depositing into the pool) is paid for in the same asset the protocol uses.

3. Working MVP

LIVE APPLICATION

https://float-arc.vercel.app

GITHUB REPOSITORY

github.com/PigAssasin/float-arc

DEPLOYED CONTRACTS ON ARC TESTNET

Contract	Address	Version
FloatCore	`0x336Be2095425ac463c6E121461B68401c3209c85`	v4
FloatPool (ERC20 fLP)	`0x98bF7f0572f542fBD6365531D39C657779839375`	v4
USDC	`0x3600000000000000000000000000000000000000`	Arc native

MVP FEATURE CHECKLIST

- **Seller dashboard:** create invoices, view on-chain credit score and advance tier, cancel timed-out invoices
- **Buyer dashboard:** approve or reject invoices, lock collateral (USDC approve + lockCollateral), pay full or partial, trigger defaults after grace period
- **Investor dashboard:** deposit USDC, withdraw, view fLP balance and live share value (rises as fees accrue)
- **Circle Wallet integration:** PIN-based wallet connection; Circle users can view all dashboard data without MetaMask
- **Float Assistant:** DeepSeek V3 AI chatbot with 4 live on-chain tool calls; answers are always current, never static
- **Legacy pool migration banner:** guides investors from the v3 pool address to v4

4. Protocol Design

Advance Rate Tiers (on-chain, not manual)

Tiers are determined by `sellerAdvanceBps(address)` on-chain, not from a raw score look-up. This matters: a new seller with no invoice history has a raw score of 50 (which would naively map to the Fair tier at 80%), but the contract correctly returns 75% (New tier) because `sellerTotalCount == 0`. This prevents Sybil attacks via small-invoice score inflation.

Tier	Credit Score	Advance Rate	Seller Stake	Buyer Collateral
New	0–40 or no history	75%	10%	25%
Fair	41–70	80%	8%	20%
Good	71–85	84%	6%	16%
Excellent	86–100	88%	5%	12%

Three-Layer Default Protection

LP principal is the last resort, not the first. When a buyer defaults, losses are absorbed in this order:

1. **Buyer collateral** (12-25% of invoice): locked at invoice approval, first to be slashed
2. **Seller stake** (5-10%): withheld from the advance disbursement, slashed if collateral is insufficient
3. **Insurance reserve:** funded by a 1% fee on every invoice; capped at 5% of pool TVL per event
4. **LP principal:** only touched if all three layers are exhausted

Credit Scoring

Seller credit score is computed deterministically on-chain:

```
sellerScore = (sellerPaidCount / sellerTotalCount) × 100
```

The score is transparent, auditable, and manipulation-resistant. It improves with each on-time repayment and falls on default. Buyers have a parallel credit record (`buyerPaidCount / buyerTotalCount`) for future governance use.

Partial Repayment

Buyers can call `payPartial(invoiceId, amount)` at any time before the due date. Partial payments reduce outstanding obligation and allow flexible cash-flow management without requiring full lump-sum repayment. The contract tracks `amountPaid` per invoice.

5. Architecture

TECHNOLOGY STACK

Layer	Technology
Blockchain	Arc Testnet, Chain ID 5042002, USDC as gas token
Smart Contracts	Solidity 0.8.20, Hardhat, OpenZeppelin (ReentrancyGuard, Ownable, SafeERC20)
Frontend Framework	Next.js 14 (App Router), TypeScript, Tailwind CSS
Blockchain Reads	wagmi v2 + viem, `useReadContracts` for batched calls
Wallet (EOA)	wagmi v2, EIP-6963 multi-wallet detection (MetaMask, OKX, etc.)

Layer	Technology
Wallet (Circle)	`@circle-fin/w3s-pw-web-sdk`, PIN-secured User-Controlled EOA
AI Assistant	DeepSeek V3, Server-Sent Events (SSE), Next.js Edge runtime
Animations	Framer Motion
Hosting	Vercel (float-arc.vercel.app)

6. Smart Contract Security

ReentrancyGuard on all writes | Checks-Effects-Interactions | Custom errors (gas efficient)

MAX_INVOICE_BPS = 20% cap | Inflation guard (1,000 dead shares)

Bounded insurance (5% of TVL) | Anti-Sybil hooks (opt-in) | One-way core registration

APPROVAL_TIMEOUT = 72h | COLLATERAL_TIMEOUT = 48h | GRACE_PERIOD = 7 days

7-day grace before default

- **MAX_INVOICE_BPS (20%):** a single invoice's advance cannot exceed 20% of available pool liquidity, preventing a single large invoice from draining the pool.
- **Inflation guard:** 1,000 dead shares are minted to the zero address on pool initialization, closing the ERC4626 share inflation attack vector.
- **Anti-Sybil hooks:** `verificationRequired` and `maxOutstandingPerSeller` flags are configurable by the operator. Both are *off by default* on testnet for open access; production can plug in EAS attestations.
- **GRACE_PERIOD (7 days):** prevents premature default triggering; buyers have a full week after due date before `markDefault` can be called.
- **Timeout cancellations:** sellers can recover stale invoices via `cancelApprovalTimeout` (72h) and `cancelCollateralTimeout` (48h), keeping the invoice set clean.

7. Float Assistant (AI)

Float Assistant is an in-app AI chatbot powered by DeepSeek V3. Unlike a static FAQ, the assistant has four live on-chain tool calls that it can invoke at any time to fetch real data before answering:

Tool	On-chain reads
`get_pool_stats`	`totalAssets()`, `availableLiquidity()`, `insuranceReserve()`, `shareValue()`
`get_my_score`	`sellerScore(address)`, `sellerAdvanceBps(address)`, paid/total counts
`get_my_invoices`	Batch reads all invoices, filters by role (seller or buyer)
`get_invoice_detail`	Full invoice struct: amount, advance, collateral, stake, status, timestamps

Streaming responses are delivered via Server-Sent Events (SSE) from a Next.js Edge runtime API route. The assistant detects the user's current dashboard (seller / buyer / investor) and tailors context accordingly, including the user's actual credit score, advance rate, and invoice history.

8. Circle Integration: Technical Deep Dive

Connection Flow

```
User clicks "Circle Wallet"
        ↓
POST /api/circle/init-user
    POST /v1/w3s/users           { userId }  →  409 if exists (not an error)
    POST /v1/w3s/users/token     { userId }  →  { userToken, encryptionKey }
        ↓
GET /api/circle/wallets  (X-User-Token: userToken)
    GET /v1/w3s/wallets
        ↓
    Wallet LIVE?  →  YES  →  return address → dashboard connected
                  →  NO   →
                        POST /api/circle/create-wallet
                            POST /v1/w3s/user/wallets  →  { challengeId }
                        sdk.execute(challengeId, callback)
                            → Circle PIN UI renders in-app
                        GET /v1/w3s/wallets  →  fetch new address
                        dashboard connected
```

useAppWallet(): Unified Wallet Hook

A custom React hook (`useAppWallet()`) unifies the wagmi and Circle connection states. All dashboards call this single hook instead of wagmi's `useAccount()` directly. This means Circle users can view their invoices, credit score, and pool stats without a browser wallet extension; write operations (which still require a signed transaction) show a clear, friendly notice if only a Circle wallet is connected.

```
useAppWallet() returns:
  { address, isConnected, isCircle }

  wagmi connected  →  address = wagmi address,  isCircle = false
  Circle connected →  address = Circle address, isCircle = true
  neither          →  address = undefined,       isConnected = false
```

9. Product Feedback

Circle: User-Controlled Wallets (W3S SDK)

WHAT WORKED WELL

- The PIN-based flow is the right UX primitive for non-crypto SME users. Eliminating seed phrases removes the single largest onboarding drop-off point in Web3.
- `sdk.execute(challengeId, callback)` is clean and composable with async/await wrappers, straightforward to integrate into a React flow.
- The `POST /v1/w3s/users` idempotency behavior (409 = already exists) is elegant for stateless re-authentication on page refresh.
- Wallet addresses are deterministic per `userId`, making testing and debugging reproducible across sessions.

SUGGESTIONS

- **wagmi connector:** an official wagmi custom connector wrapping Circle SDK would enable full transaction signing (not just reads) without building a parallel signing pipeline. This is the most impactful missing piece for DeFi integrations.
- **Passkey / WebAuthn support:** as an alternative to PIN, especially for mobile users where PIN entry is cumbersome.
- **Lightweight read-only SDK:** a version that does not require DOM mounting would simplify server-side rendering contexts and reduce bundle size for read-heavy apps.
- **TypeScript types for sdk.execute callback:** the `result.type` values are underdocumented; a discriminated union type would improve developer experience significantly.

Arc Testnet

WHAT WORKED WELL

- USDC as native gas token is the standout feature for stablecoin applications; users never need to acquire a separate asset just to pay transaction fees.
- Full EVM compatibility meant zero tooling changes: Hardhat, wagmi v2, viem, and ethers.js all worked out of the box.
- The `defineChain` pattern in wagmi slots in cleanly for Arc's custom chain ID.

SUGGESTIONS

- **Additional public RPC endpoint:** occasional timeout spikes during development were resolved with a fallback RPC, but a second officially supported endpoint would reduce this friction.
- **Block explorer event search:** arcscan.app is functional but lacks event log filtering, making it harder to debug contract interactions without a custom indexer.
- **Chainlist / MetaMask Snap entry:** Chain ID 5042002 is not in common wallet presets, requiring users to add the network manually. An entry on Chainlist or a MetaMask Snap would significantly reduce onboarding friction.
- **Official testnet USDC faucet UI:** a faucet with a simple web UI (rather than requiring script-based minting) would accelerate developer onboarding considerably.